MEGA2560 as AND gate for RS232 – Dokumentacja

Bartosz Włodarczyk

15 października 2026

Spis treści

1 Opis zainicjalizowanych portów szeregowych w MEGA2560 1

2 Struktura przechowująca otrzymane wiadomości 2

3 Główna pętla programu 2

4 Funkcji trybu: „AND” 3

1 Opis zainicjalizowanych portów szeregowych w MEGA2560

Kolorem niebieskim zaznaczono normalnie działające porty szeregowe.

Port: `HardwareSerial`

`Serial` — PC_USB (Tx0 pin1; Rx0 pin0)
Interface do komunikacji przez złącze USB-B: debug oraz programowanie MEGA2560.
[Nie używany w czasie normalnej pracy[a]]

[a] Piny 0 i 1 nie obsługują w pełni poprawnie komunikacji: uniemożliwiając podłączenie ich do konwertera RS232-UART, co pozwala tylko na połączenie przez port USB-B.

`Serial1` — HF_Sensor (Tx1 pin18; Rx1 pin19)
Interface do komunikacji z czujnikiem HF.

`Serial2` — HF_PC (Tx2 pin16; Rx2 pin17)
Interface do komunikacji z maszyną produkcyjną z portem odpowiedzialnym za komunikację od czujnika HF.

`Serial3` — UHF_Sensor (Tx3 pin14; Rx3 pin15)
Interface do komunikacji z czujnikiem UHF.

Port: `SoftwareSerial`

`Serial_4` — UHF_PC (Tx1 pin51; Rx1 pin50)
Interface do komunikacji z maszyną produkcyjną z portem odpowiedzialnym za komunikację od czujnika UHF.
[Nie używany[a]]

[a] Awaria konwertera RS232-UART uniemożliwia na stworzenie piątego kanały komunikacji, który był zarezerwowany na komunikację MEGA2560-UHF_PC. Do płytki MEGA podłączone są 4 sprawne konwertery i jeden popsuty.

`Serial_5` — AppPort (Tx1 pin53; Rx1 pin52)
Interface do komunikacji z aplikacją HMI.

2 Struktura przechowująca otrzymane wiadomości

Otrzymane wiadomości przechowywane są w strukturze, zawierającej informacje w jakim czasie od rozpoczęcia pracy MEGA2560 dana wiadomość nadeszła, umożliwiając odrzucanie wiadomości, które przekroczą określony `FRAME_TIMEOUT`.

```
12 struct serialData {
13   unsigned mesLength = 0;       // Długość wiadomości
14   unsigned long mesTime = 0;    // Stępel czasowy otrzymania wiadomości
15   uint8_t buffer[BUFFER_SIZE]; // Wiadomość
16 };
```

HexFun.h

```
8  #define BUFFER_SIZE 128        // Wielkość wiadomości
9  #define FRAME_TIMEOUT 100      // [ms]
10 #define FRAME_UHF_TIMEOUT 20   // [ms]
11 #define UHF_MESSAGE_LENGTH 8   // Oczekiwana długość wiadomości UHF w [bajtach]
12 #define zapytanie_HF_LENGTH 7 // Długość zapytania wiadomości HF 0x07
13 #define DEFAULT_DISTANCE 6     // Liczba anten pomiędzy czyjnikami + 2
14
15 #define BOUND_UHF 115200
16 #define BOUND_HF 38400
```

config.h

3 Główna pętla programu

Fragment zakomenderowany programu odpowiedzialny za obsługę płytki z możliwością przełączania między trybem pracy łączącym sygnały od UHF i HF („AND") oraz trybem pracy podsłuchującym („SNIFER"), gdzie program przesyła sygnały miedzy czujnikiem a maszyną/komputerem bez ingerencji w nie. Część programu porzucona przez awarię konwertera RS232-UART zmniejszającą liczbę dostępnych postów szeregowych, uniemożliwiając podłączenie z wejściem maszyny od portu UHF.

```
60 void loop() {
61   if(workAsSnifer) { // (TRYB: SNIFER)
62     handleSerialPort_HF(  HF_Sensor,  HF_PC,     serialData1, "S01");
63     handleSerialPort_HF(  HF_PC,      HF_Sensor, serialData2, "S02");
64     handleSerialPort_UHF(UHF_Sensor, UHF_PC,     serialData3, "S03");
65     handleSerialPort_UHF(UHF_PC,     UHF_Sensor, serialData4, "S04");
66   } else { // (TRYB: AND)
67     handleSerialPort_HF_CZUJNIK( HF_Sensor,  HF_PC,                 serialData1, "01");
68     handleSerialPort_HF_PC(      HF_PC,     UHF_Sensor, HF_Sensor, serialData2, "02");
69     handleSerialPort_UHF_CZUJNIK(           UHF_Sensor, HF_PC,     serialData3, "03");
70     /* Port 04 (UHF) jest ingnorowany w tym trybie pracy */
71     czujnikiANDgate(HF_PC, HF_Sensor, AppPort);
72   }
73   // handleSerialPortCommunication(PC_USB, HF_PC);
74   handleSerialPortCommunication(AppPort, HF_PC);
75   // debugCommunication(PC_USB);
76 }
```

Prototyp4.ino

Program zastąpiono wersją jedynie pracującą w trybie „AND”.

```
80 void loop() { /* Z powodu problemów HARDWAREowych program może działać tylko jako AND gate
81   handleSerialPort_HF_CZUJNIK( HF_Sensor,  HF_PC,                 serialData1, "01");
82   handleSerialPort_HF_PC(      HF_PC,     UHF_Sensor, HF_Sensor, serialData2, "02");
83   handleSerialPort_UHF_CZUJNIK(           UHF_Sensor, HF_PC,     serialData3, "03");
84   /* Port 04 (UHF) jest ingnorowany w tym trybie pracy */
85   czujnikiANDgate(HF_PC, HF_Sensor, AppPort);
86
87   // handleSerialPortCommunication(PC_USB, HF_PC);
88   handleSerialPortCommunication(AppPort, HF_PC);
89   // debugCommunication(PC_USB);
90 }
```

Prototyp4.ino

4 Funkcji trybu: „AND”

Obsługa komunikacji (HF czujnik-MEGA2560) – `handleSerialPort_HF_CZUJNIK`

Parametry:

- `HardwareSerial& readingSerial` – Port czujnika HF (otrzymywanie wiadomości);
- `HardwareSerial& writingSerial` – Port maszyny HF (wysyłanie wiadomości));
- `serialData& sd` – struktura przechowująca treść wiadomości;
- `const char* SerialNum` – Nr. portu do wydruku w konsoli (debug);

Timeout jest liczony względem pierwszego nadesłanego bajtu.

Działanie: Jeżeli nowe wiadomości dostępne:

1. Jeżeli część niepełnej wiadomości nadeszła: dawniej niż czas `FRAME_TIMEOUT` odrzuć poprzedni fragment wiadomości.
2. Jeżeli nie wczytano jeszcze żadnego fragmentu wiadomości: wczytaj długość nowej wiadomości z pierwszej bajtu i zapisz czas przybycia pierwszego bajtu.
3. Jeżeli reszta wiadomości przybyła: wczytaj ją całą do struktury wiadomości. Zinterpretuj otrzymaną wiadomość.

Obsługa komunikacji (HF czujnik-MEGA2560) – `handleSerialPort_HF_CZUJNIK`

Parametry:

- `HardwareSerial& readingSerial` – Port czujnika HF (otrzymywanie wiadomości);
- `HardwareSerial& writingSerial` – Port maszyny HF (wysyłanie wiadomości));
- `serialData& sd` – struktura przechowująca treść wiadomości;
- `const char* SerialNum` – Nr. portu do wydruku w konsoli (debug);

Timeout jest liczony względem ostatniego otrzymanego bajtu.

Działanie: Jeżeli nowe wiadomości dostępne:

1. Jeżeli część niepełnej wiadomości nadeszła: dawniej niż czas `FRAME_TIMEOUT` odrzuć poprzedni fragment wiadomości.
2. Jeżeli dalej całość wiadomości nie nadeszła: zapisz czas przybycia wczytywanego bajtu, wczytaj do buffora najnowszy bajt wiadomości oraz wydłuż długość wiadomości.
3. Jeżeli długość wiadomości jest ≥ 8 (Gdzie 8 bajtów to długość całej wiadomości w komunikacji UHF) zinterpretuj trzecią wiadomość (zawierającą informacje czy antena jest dobra czy zła), (pozostałe wiadomości, tzn. 1, 2 i 4 są tylko sprawdzane czy nie są kodami błędów).

Obsługa podwójnego sprawdzania anteny – `BAD_UHF_CZUJNIK`

Funkcja wywoływana po zinterpretowaniu złej anteny UHF. Funkcja wysyła do czujnika UHF (`HardwareSerial& readingSerial`) ponowne żądanie sprawdzenia anteny w przypadku jeżeli to było pierwsze badanie. Jeżeli było to drugie sprawdzanie tej samej anteny funkcja oznacza antenę jako złą.

Działanie:

1. (Pierwszy raz oznaczono antenę jako złą) Program wysyła polecenie do UHF sprawdzenia anteny oraz nie ustawia flagi otrzymania wiadomości od czujnika (`UHF_flag`), co powoduje że program dalej czeka na drugą odpowiedz.
2. (Drugi raz oznaczono antenę jako złą) Program ustawia flagę (`UHF_flag`) informując że stan anteny jest już znany.

Mechanizm ponownego sprawdzania anteny UHF został zaimplementowany w funkcji obsługującej komunikację przez wzgląd na to że antena UHF jest sprawdza w „przeszłości" względem sprawdzania tej samej anteny przez czujnik HF.
W momencie stwierdzenia anteny jako złej — przez badanie czujnikiem UHF — w trakcie badania czujnikiem HF, nie było by już możliwości powtórzenia badania czujnikiem UHF.

Obsługa komunikacji (HF komputer-MEGA2560) – `handleSerialPort_HF_PC`

Parametry:

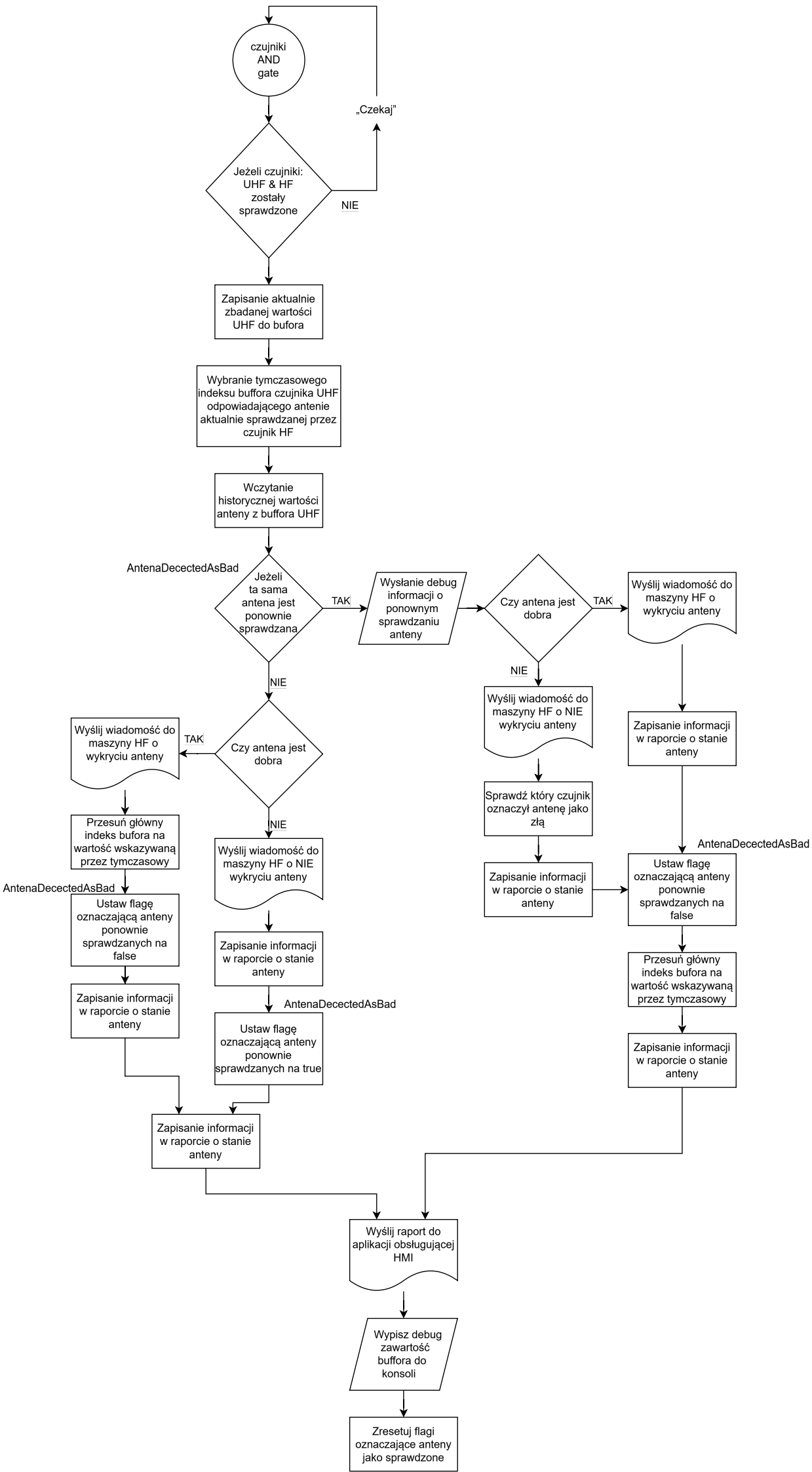
- `HardwareSerial& readingSerial` – Port czujnika HF (otrzymywanie wiadomości);
- `HardwareSerial& writing_UHF_Serial` – Port maszyny UHF (wysyłanie wiadomości));
- `HardwareSerial& writing_HF_Serial` – Port maszyny HF (wysyłanie wiadomości));
- `serialData& sd` – struktura przechowująca treść wiadomości;
- `const char* SerialNum` – Nr. portu do wydruku w konsoli (debug);

Timeout jest liczony względem pierwszego nadesłanego bajtu.

Działanie: Jeżeli nowe wiadomości dostępne:

1. Jeżeli część niepełnej wiadomości nadeszła: dawniej niż czas `FRAME_TIMEOUT` odrzuć poprzedni fragment wiadomości.
2. Jeżeli nie wczytano jeszcze żadnego fragmentu wiadomości: wczytaj długość nowej wiadomości z pierwszej bajtu i zapisz czas przybycia pierwszego bajtu.
3. Jeżeli reszta wiadomości przybyła: wczytaj ją całą do struktury wiadomości. Zinterpretuj otrzymaną wiadomość
   (a) Jeżeli pytanie o stan anteny:
      i. Jeżeli HMI wysłało polecenie STOP wyślij do maszyny kod błędu;
      ii. Jeżeli Jeżeli jest to pierwszy odczyt danej anteny: wyślij polecenia sprawdzenia anten do obu czujników;
      iii. Jeżeli jest to drugi odczyt danej anteny: wyślij polecenie sprawdzenia anten tylko do czujnika HD, tylko jeżeli wg. czujnika UHF antena była dobra.
   (b) ELSE prześlij wiadomość dalej bez ingerencji w jej treść.
   (c) Wyślij wiadomość do konsoli jeżeli załączono `DEBUG_DISPLAY`.

Działanie funkcji `void czujnikiANDgate(HardwareSerial& writingSerial, HardwareSerial& writing_HF_Serial, Stream &AppSerial)` przedstawino na schemacie:


czujniki AND gate
„Czekaj”
Jeżeli czujniki: UHF & HF zostały sprawdzone
NIE
Zapisanie aktualnie zbadanej wartości UHF do bufora
Wybranie tymczasowego indeksu buffora czujnika UHF odpowiadającego antenie aktualnie sprawdzanej przez czujnik HF
Wczytanie historycznej wartości anteny z buffora UHF
AntenaDecectedAsBad
Jeżeli ta sama antena jest ponownie sprawdzana
TAK
Wysłanie debug informacji o ponownym sprawdzaniu anteny
Czy antena jest dobra
TAK
Wyślij wiadomość do maszyny HF o wykryciu anteny
NIE
Wyślij wiadomość do maszyny HF o NIE wykryciu anteny
Zapisanie informacji w raporcie o stanie anteny
Sprawdź który czujnik oznaczył antenę jako złą
Zapisanie informacji w raporcie o stanie anteny
AntenaDecectedAsBad
Ustaw flagę oznaczającą anteny ponownie sprawdzanych na false
Przesuń główny indeks bufora na wartość wskazywaną przez tymczasowy
Zapisanie informacji w raporcie o stanie anteny
NIE
Czy antena jest dobra
TAK
Wyślij wiadomość do maszyny HF o wykryciu anteny
Przesuń główny indeks bufora na wartość wskazywaną przez tymczasowy
AntenaDecectedAsBad
Ustaw flagę oznaczającą anteny ponownie sprawdzanych na false
Zapisanie informacji w raporcie o stanie anteny
NIE
Wyślij wiadomość do maszyny HF o NIE wykryciu anteny
Zapisanie informacji w raporcie o stanie anteny
AntenaDecectedAsBad
Ustaw flagę oznaczającą anteny ponownie sprawdzanych na true
Zapisanie informacji w raporcie o stanie anteny
Wyślij raport do aplikacji obsługującej HMI
Wypisz debug zawartość buffora do konsoli
Zresetuj flagi oznaczające anteny jako sprawdzone